As a Senior Backend Software Architect, I have designed the Banking Management System to prioritize **consistency, security, and auditability**. Given the financial nature of the project, we will employ a microservices-based approach with an emphasis on Event-Driven Architecture (EDA) to ensure decoupled scalability.

---

1. Architecture Overview

- **Style:** Microservices (Golang for high performance/concurrency) with a sidecar pattern for security/observability.
- **API Gateway:** Kong or Traefik for rate limiting, JWT validation, and SSL termination.
- **Communication:**

* **Synchronous:** gRPC for internal service-to-service communication.

* **Asynchronous:** Kafka for transaction events, audit logs, and fraud detection pipelines.

- **Database:** PostgreSQL (Primary) for transactional integrity (ACID compliance); Redis for session management and caching.

---

2. Services

- **Identity Service:** Manages AuthN/AuthZ, MFA secrets, and user profiles.
- **Account Service:** Manages balance, ledgers, and account status.
- **Transaction Service:** Handles fund transfers, bill payments, and idempotent transaction processing.
- **Loan Service:** Orchestrates loan lifecycle, credit score checks, and employee review workflows.
- **Fraud/AI Service:** Analyzes transaction patterns via Kafka streams; integrates with ML models for risk scoring.
- **Audit Service:** Write-once service that captures all system-wide state changes.

---

3. Modules (Internal service structure)

- **Domain:** Core business logic (Entities, Interfaces).
- **Application:** Use cases (Orchestrators).

- **Infrastructure:** Adapters (DB drivers, external API clients like Credit Bureaus).
- **Interface:** API handlers (REST/gRPC controllers).

---

4. Authentication & Authorization

- **AuthN:** OAuth2 + OIDC. MFA enforced via TOTP (Time-based One-Time Password) at the Gateway layer.
- **AuthZ:** RBAC (Role-Based Access Control) using claims-based JWTs. Sensitive actions (like loan approval) require **MFA-step-up authentication**.

---

5. Business Logic

- **Idempotency:** All financial transfers will require an `Idempotency-Key` header to prevent double-spending during network retries.
- **Distributed Transactions:** Use the **Saga Pattern** (orchestration-based) to manage multi-step processes like loan applications spanning different services.

---

6. Background Jobs

- **Engine:** Temporal.io or Go-workers.
- **Use Cases:** End-of-month budget reports, interest accrual, automated fraud report generation, and third-party credit score synchronization.

---

7. File Storage

- **Storage:** AWS S3 (encrypted with KMS).
- **Usage:** Immutable storage for KYC documentation and loan application PDFs. Presigned URLs are used for secure client access.

---

8. Error Handling

- **Standardization:** Use a unified `AppError` struct (Code, Message, CorrelationID).
- **Resiliency:** Implement Circuit Breakers (via Hystrix or Sentinel) for all third-party integrations (e.g., Credit Bureau).

---

9. Logging & Observability

- **Structured Logging:** Zap/Zerolog for JSON output.
- **Tracing:** OpenTelemetry + Jaeger for end-to-end distributed tracing.
- **Metrics:** Prometheus + Grafana for monitoring transaction throughput and latency.

---

10. Folder Structure (Standardized per service)

```
/cmd/service-name/main.go
/internal/
  /domain/          # Interfaces and models
  /usecase/         # Business logic
  /repository/      # DB implementations
  /transport/       # HTTP/gRPC handlers
/pkg/               # Shared utilities
/deployments/       # K8s manifests / Helm charts
/api/               # Protobuf definitions
```

---

11. Recommended Libraries (Go Stack)

- **Framework:** `Gin` or `Echo` (REST), `gRPC-Go` (Internal).
- **DB/ORM:** `sqlx` (for performance) or `GORM` (if developer velocity is prioritized).
- **Validation:** `go-playground/validator`.
- **Config:** `Viper`.
- **Testing:** `testify` for mocking and assertions.
- **Secrets:** HashiCorp Vault.

Implementation Tip

Since this is a banking system, **avoid "Distributed Monolith" traps.** Ensure that each service has its own database schema. For auditing, use **Event Sourcing** where every state change is recorded as an immutable event in the `Audit_Logs` database before the main entity table is updated.